

- The right-hand side of (44) comes from (42). By definition, $n_{\phi,i}$ is a node of type IMPLIES, and let $\gamma \triangleq n_{n_{\phi,i},2}$ and $\xi \triangleq n_{n_{\phi,i},1}$ denote the other neighbors of $n_{\phi,i}$. By definition, we have $\phi = \xi \rightarrow \gamma$, $\tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}} = \min(1, 1 - L(\xi) + U(\gamma))$, $p(S_\xi) \geq L(\xi)$, and $p(S_\gamma) \leq U(\gamma)$. It is straightforward to verify that

$$S_\phi = S_{\neg\xi} \cup S_\gamma \quad (82)$$

Therefore,

$$\begin{aligned} p(S_\phi) &\leq p(S_{\neg\xi}) + p(S_\gamma) \\ &= 1 - p(S_\xi) + p(S_\gamma) \\ &\leq 1 - L(\xi) + U(\gamma) \end{aligned} \quad (83)$$

Since $p(S_\phi) \leq 1$, it must be true that

$$p(S_\phi) \leq \min(1, 1 - L(\xi) + U(\gamma)) = \tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}} \quad (84)$$

Hence (44) is true in this scenario. $\square$

Now we are ready to present the main proof.

Proof of Theorem 2. Let L^k and U^k denote the L and U functions in the pseudo code after k iterations. Define $\Gamma^k = \{(v, L^k(v), U^k(v)) \mid v \in V_1\}$ for $k = 0, 1, \dots$. By the pseudo code, it is straightforward to verify that $P_{\Gamma^0} = P_{\Gamma_0}$. By Lemma 1, we have $P_{\Gamma^k} = P_{\Gamma^{k+1}}$ for any k . Therefore, after convergence it must be true that $P_{\Gamma^k} = P_{\Gamma_0}$.

By definition, for any $p \in P_{\Gamma^k}$ after convergence, we have

$$L_\sigma = L^k(\sigma) \leq p(S_\sigma) \leq U^k(\sigma) = U_\sigma \quad (85)$$

Replacing P_{Γ^k} with P_{Γ_0} , we get the two inequalities in Theorem 2. $\square$

E Learning with constraints

g Here we present a slightly expanded discussion of the constrained optimization problem discussed in Section 6.

E.1 Constraints

Constraints on neural parameters are derived from the truth tables of the operations they intend to model and from established ranges for “true” and “false” values. Given a threshold of truth $\frac{1}{2} < \alpha \leq 1$, a continuous truth value is considered true if it is greater than α and false if it is less than $1 - \alpha$. Accordingly, the truth table for, e.g., binary AND suggests a set of constraints given

p	q	$p \wedge q$		p	q	$p \wedge q$		$\beta - w_p \cdot (1 - p) - w_q \cdot (1 - q)$	
F	F	F		$1 - \alpha$	$1 - \alpha$	$1 - \alpha$		$\beta - w_p \cdot \alpha - w_q \cdot \alpha$	$\leq 1 - \alpha$
F	T	F	$\rightarrow$	$1 - \alpha$	1	$1 - \alpha$	$\rightarrow$	$\beta - w_p \cdot \alpha$	$\leq 1 - \alpha$
T	F	F		1	$1 - \alpha$	$1 - \alpha$		$\beta - w_q \cdot \alpha$	$\leq 1 - \alpha$
T	T	T		α	α	α		$\beta - w_p \cdot (1 - \alpha) - w_q \cdot (1 - \alpha)$	$\geq \alpha$

More generally, n -ary conjunctions have constraints of the form

$$\begin{aligned} \forall i \in I, \quad w_i &\geq 0 \\ \forall i \in I, \quad \beta - w_i \cdot \alpha &\leq 1 - \alpha \end{aligned} \quad (86)$$

$$\beta - \sum_{i \in I} w_i \cdot (1 - \alpha) \geq \alpha \quad (87)$$

while n -ary disjunctions have constraints of the form

$$\begin{aligned} \forall i \in I, \quad w_i &\geq 0 \\ \forall i \in I, \quad 1 - \beta + w_i \cdot \alpha &\geq \alpha \end{aligned} \quad (88)$$

$$1 - \beta + \sum_{i \in I} w_i \cdot (1 - \alpha) \leq 1 - \alpha \quad (89)$$

The identity $p^{(w_p)} \rightarrow^\beta q^{(w_q)} = (\neg p)^{(w_p)} \oplus^\beta q^{(w_q)}$ permits implications to use the same constraints as disjunctions and, in fact, the above two sets of constraints are equivalent under the De Morgan laws. A consequence of these constraints is that LNN evaluation is guaranteed to behave classically, i.e. to yield results at every neuron within the established ranges for true and false, if all of their inputs are themselves within these ranges.

E.2 Slack variables

It is easy to see that weights w_i cannot equal 0 under the above constraints, though this is a desirable outcome of optimization as it effectively removes the affected input. To permit this, it is necessary to introduce a slack variable for each weight, allowing its respective constraints in (86) or (88) to be violated as the weight drops to 0:

$$\begin{aligned} \forall i \in I, \quad & s_i \geq 0 \\ \forall i \in I, \quad & \beta - w_i \cdot \alpha - s_i \leq 1 - \alpha \quad (86^*) \\ \forall i \in I, \quad & 1 - \beta + w_i \cdot \alpha + s_i \geq \alpha \quad (88^*) \end{aligned}$$

If $w_i = 0$ is understood as $i \notin I$, this update remains consistent with the original constraint if either $s_i = 0$ or $w_i = 0$. One can encourage optimization to favor such parameterizations by included in the loss function a penalty term that scales with $s_i w_i$. The coefficient on this penalty term controls how classical learned operations must be, with exact classical behavior restored if optimization reduces the penalty term to 0.

E.3 Optimization problem

The optimization problem from before is then updated

$$\begin{aligned} \min_{B, W, S} \quad & E(B, W) + \sum_{k \in N} \max\{0, L_{B, W, k} - U_{B, W, k}\} + \sum_{k \in N} \mathbf{s}_k \cdot \mathbf{w}_k \\ \text{s.t.} \quad & \forall k \in N, i \in I_k, \quad \alpha \cdot w_{ik} - s_{ik} - \beta_k + 1 \geq \alpha, \quad w_{ik}, s_{ik} \geq 0 \\ & \forall k \in N, \quad \sum_{i \in I_k} (1 - \alpha) \cdot w_{ik} - \beta_k + 1 \leq 1 - \alpha, \quad \beta_k \geq 0 \end{aligned}$$

for loss function E , bias vector B , weight matrix W , slack matrix S , neuron index set N , and inferred lower and upper bounds $L_{B, W, k}$ and $U_{B, W, k}$ at each neuron. In addition, it may be of interest to square either or both of the contradiction penalty terms $\max\{0, L_{B, W, k} - U_{B, W, k}\}$ or the slack penalty terms $s_i w_i$.

Depending on the specific problem being solved, different loss functions $E(B, W)$ may be used. For example, an LNN configured to predict a binary outcome may use mean squared error as usual. Alternately, it is possible to use the contradiction penalty to build arbitrarily complex logical loss functions by introducing new formulae into model that become contradictory in the event of undesirable inference behavior. Understandably, the parameters of specifically these introduced formulae should not be tuned but instead left in a default state (e.g. all 1), so optimization cannot turn the logical loss function off.

F Tailored activation function

F.1 Motivation

In weighted fuzzy logic, a logical operation on inputs is carried out by a neuron activation function, in almost the same way that artificial neurons compute on their inputs. The main difference is that the weights and bias terms are constrained in such a way that the semantics of the operation are maintained. For example, Equation (6) describes how at least one input to a disjunction neuron may be True with the rest False, results in the activation function returning a value less than or equal to α , representing classically True.

The constraining of weights and biases, ensures that while parameters can be adjusted (partially), the neuron retains its logical character and its interpretability.

Controlling the allowed weights and biases of the inputs enforces the correct logical operation after applying the static activation function. These constraints define what the activation function is going to do the weighted sum.

A more natural way to control the output of the activation function is to change the activation function itself and in this way enforce the right logical output. This approach has no constraints on the weights and all the enforcement is effected by having an activation function that depends on the weights themselves. We call this the ‘tailored activation function’ approach. It builds on the weighted fuzzy logic approach and is in-fact equivalent to weighted Łukasiewicz logic in the $\alpha = 1$, equally-weighted case.

There are many advantages to the tailored activation function approach. To unpack this let us re-look at the problems with the constrained approach:

- The constrained approach dictates that the activation function must be α -preserving to provide interpretability on the domain. This fixes two static points that the activation function must go through, prematurely giving static truth meaning to the values of the weighted sum.
- Additional slack parameters are required to relax constraints as weights approach zero — hindering interpretability and encouraging over-fitting.
- Parameter updates require a constraint satisfaction algorithm — such as Frank Wolfe or Projected Gradient Descent.
- Constraint satisfaction and neuron operations are expensive, with slacks required for each weight and logical operations defined $\forall i w_i, s_i$ in (86*), (88*).
- Unbounded weights become unwieldy and less interpretable.
- There are no gradients in the clamped regions without gradient transparent clamping, as in Appendix G.
- The $w_i \cdot (1 - x_i)$ form of a weighted conjunction in (10) prevents maximally True inputs ($x_i = 1$) from offering gradients — this is problematic for contradictory losses that exist when given a False conjunction that has all True inputs. This complication extends to disjunction and implication neurons accordingly.
- α is not interpretable especially in choosing its value.
- β is not easily interpretable.
- β must be learnt for each neuron — which encourages over-fitting.
- downward inference from functional inverses compute classically incorrect bounds.

F.2 Bounds

Here we present the expanded table of LNN bounds presented in section 3 to better elaborate on the tailored activation formulation.

The interpretation of lower and upper bound truth values correspond to one of 4 primary or 3 secondary states that a neuron can be in:

Table 3: Primary and secondary truth value bound states

State	Primary		State	Secondary	
	Lower	Upper		Lower	Upper
Unknown	$[0, 1 - \alpha]$	$[\alpha, 1]$	$\sim$ Unknown	$[0, 0.5]$	$[0.5, 1]$
True	$[\alpha, 1]$	$[\alpha, 1]$	$\sim$ True	$(0.5, \alpha)$	$(0.5, 1]$
False	$[0, 1 - \alpha)$	$[0, 1 - \alpha]$	$\sim$ False	$[0, 0.5)$	$(1 - \alpha, 0.5)$
Contradiction	Lower	> Upper			

Neurons with a True or False state may be considered classically-true or classically-false respectively. A $\sim$ True or $\sim$ False state is interpreted as being more-true-than-not or more-false-than-not, while still remaining open to a classically-true or classically-false convergence provided an upper bound above α or lower bound below $1 - \alpha$ respectively. The Unknown state, albeit non-classical, may converge to a classical state, whereas $\sim$ Unknown may converge to be $\sim$ True or $\sim$ False but not to a classical state. A Contradictory state represents a disagreement in assertions of the truth between connected rule and fact neurons, with intra-classical contradictions typically being ignored.